IMPLEMENTATION OF CLIENT SCRIPTS AND UI POLICIES IN INCIDENT APPLICATION USING SERVICENOW

PROJECT DESIGN PHASE

1. INTRODUCTION

ServiceNow is a cloud-based platform used by organizations to manage IT services, business processes, incidents, requests, changes, and other operational activities. It provides a centralized platform where organizations can automate workflows and improve the efficiency of their service management processes.

The Incident Management module in ServiceNow is used to record, track, assign, and resolve incidents reported by users. An incident form contains several fields such as Caller, Category, Subcategory, Impact, Urgency, Priority, Assignment Group, Assigned To, Short Description, and Additional Comments.

In a standard ServiceNow environment, users may enter incorrect or incomplete information while creating or updating incidents. To improve data accuracy and provide a better user experience, Client Scripts and UI Policies can be implemented.

Client Scripts are used to perform client-side actions when a form is loaded, submitted, changed, or validated. UI Policies are used to dynamically control the behavior of form fields, such as making fields mandatory, visible, or read-only.

This project focuses on implementing Client Scripts and UI Policies specifically for the Incident application. The project demonstrates how form behavior can be customized according to organizational requirements.

2. PROJECT PLANNING PHASE

The project planning phase is the initial stage of the project. It defines the project objectives, requirements, scope, resources, activities, and expected outcomes.

Proper planning helps ensure that the implementation is completed systematically and that all required ServiceNow configurations are tested before deployment.

2.1 Project Title

Implementation of Client Scripts and UI Policies in Incident Application Using ServiceNow

2.2 Project Objective

The main objective of this project is to customize the Incident form in ServiceNow using Client Scripts and UI Policies.

The specific objectives are:

To understand Incident Management in ServiceNow.

To understand the purpose and functionality of Client Scripts.

To understand the purpose and functionality of UI Policies.

To improve data validation on the Incident form.

To dynamically control the visibility of Incident fields.

To make selected fields mandatory based on specific conditions.

To make fields read-only when required.

To reduce incorrect or incomplete incident information.

To improve the overall user experience of the Incident form.

To test the implemented configurations in a ServiceNow environment.

2.3 Problem Statement

When incidents are created manually, users may provide incomplete, incorrect, or inconsistent information. Some fields may need to be mandatory only under certain conditions, while other fields may need to be hidden or made read-only depending on the incident information.

For example, if an incident is assigned to a particular group, additional information may be required. Similarly, certain fields may not be relevant for every incident and can be hidden from the user.

Without proper form controls, this can result in:

Incomplete incident records.

Incorrect data entry.

Unnecessary fields being displayed.

Increased manual verification.

Inconsistent incident information.

Reduced efficiency in incident processing.

Therefore, Client Scripts and UI Policies are implemented to provide dynamic form behavior and improve data quality.

3. PROJECT SCOPE

The scope of this project is limited to the Incident application in ServiceNow.

The project includes:

Studying the Incident Management module.

Identifying fields that require customization.

Creating Client Scripts.

Creating UI Policies.

Configuring field behavior.

Implementing client-side validation.

Making fields mandatory based on conditions.

Controlling field visibility.

Controlling field read-only behavior.

Testing the implemented configurations.

The project does not focus on complete Incident Management implementation, advanced integrations, or server-side business rules.

4. PROJECT REQUIREMENTS

4.1 Functional Requirements

The system should:

Allow users to create and update incidents.

Validate Incident form information.

Dynamically change field behavior.

Display relevant fields based on conditions.

Make required fields mandatory when necessary.

Make selected fields read-only when required.

Provide appropriate messages when invalid information is entered.

Prevent submission when required validation conditions are not satisfied.

4.2 Non-Functional Requirements

The system should:

Be easy to use.

Provide quick client-side responses.

Maintain data accuracy.

Be reliable.

Be easy to maintain.

Follow ServiceNow configuration standards.

Avoid unnecessary customization.

5. HARDWARE AND SOFTWARE REQUIREMENTS

5.1 Hardware Requirements

Component	Requirement
Processor	Intel Core i3 or above
RAM	Minimum 4 GB
Storage	Minimum 20 GB free space
Internet	Stable Internet connection
Display	Standard monitor/laptop display

5.2 Software Requirements

Software	Purpose
ServiceNow	Development and testing platform
Web Browser	Accessing ServiceNow
GitHub	Project documentation and version management

VS Code Optional script editing/documentation

Windows/Linux/macOS Operating environment

6. TECHNOLOGIES USED

ServiceNow

ServiceNow is the primary platform used to develop and test the project.

Client Scripts

Client Scripts are JavaScript-based scripts executed on the client side of ServiceNow forms. They can respond to events such as:

Form Load

Field Change

Form Submission

UI Policies

UI Policies control the behavior of fields on ServiceNow forms without requiring complex scripting for basic field behavior.

They can control:

Mandatory state

Visible state

Read-only state

JavaScript

JavaScript is used when custom client-side logic is required through Client Scripts.

GitHub

GitHub is used to store the project source code, documentation, screenshots, and other project-related files.

7. PROJECT PLANNING ACTIVITIES

The project was divided into several activities to ensure systematic development.

Phase 1 – Requirement Analysis

The Incident application requirements were studied and the fields requiring dynamic behavior were identified.

Phase 2 – ServiceNow Environment Setup

The ServiceNow instance was accessed and the Incident application was examined.

Phase 3 – Incident Form Analysis

The existing Incident form and its important fields were studied.

Phase 4 – Client Script Planning

The conditions requiring client-side validation and dynamic behavior were identified.

Phase 5 – UI Policy Planning

Fields requiring mandatory, visible, or read-only behavior were identified.

Phase 6 – Implementation

Client Scripts and UI Policies were created and configured.

Phase 7 – Testing

Different Incident form scenarios were tested to verify whether the configurations worked correctly.

Phase 8 – Documentation

Screenshots, configurations, results, and project details were documented for the final report.

8. PROJECT IMPLEMENTATION PLAN

The implementation follows the sequence:

Requirement Analysis

↓

Incident Form Study

↓

Identify Required Fields

↓

Plan Client Scripts

↓

Plan UI Policies

↓

Create Client Scripts

↓

Create UI Policies

↓

Test Incident Form

↓

Fix Errors

↓

Final Validation

↓

Documentation

9. CLIENT SCRIPT IMPLEMENTATION

Client Scripts are used to control Incident form behavior using JavaScript.

ServiceNow provides four main types of Client Scripts:

9.1 onLoad

The script executes when the Incident form is loaded.

Purpose:

Set default values.

Display messages.

Change field behavior when the form opens.

9.2 onChange

The script executes when the value of a selected field changes.

Purpose:

Dynamically modify other fields.

Validate entered information.

Display or hide related information.

9.3 onSubmit

The script executes when the user submits the form.

Purpose:

Validate information before submission.

Prevent submission when required conditions are not satisfied.

9.4 onCellEdit

This is used for client-side validation when editing records directly from a list.

10. SAMPLE CLIENT SCRIPT REQUIREMENT

One example requirement for the Incident application is to validate the Short Description field before submitting an incident.

If the Short Description is empty, the system should display an error message and prevent submission.

Example logic

```
function onSubmit() {
   var shortDescription = g_form.getValue('short_description');

   if (shortDescription == '') {
      g_form.addErrorMessage('Short Description is required.');
      return false;
   }

   return true;
}
```

This validation improves data quality by preventing an Incident from being submitted without a meaningful description.

11. UI POLICY IMPLEMENTATION

UI Policies provide a simple way to dynamically control fields on the Incident form.

A UI Policy can make a field:

Mandatory

Visible

Read-only

depending on a specified condition.

Example

Suppose the requirement is:

When the Incident State is Resolved, the Resolution Code field should become mandatory.

The UI Policy can be configured with:

Table: Incident

Condition: State is Resolved

UI Policy Actions: Resolution Code → Mandatory

This ensures that resolution information is entered before the incident is completed.

12. UI POLICY ACTIONS

UI Policy Actions determine what happens to individual fields when the UI Policy condition is satisfied.

Mandatory

The field must contain a value before the form can be submitted.

Visible

The field is displayed or hidden based on the configured condition.

Read-only

The field can be prevented from being edited by the user.

13. EXAMPLE PROJECT CONFIGURATIONS

The following configurations can be included as examples in the project.

Configuration	Condition	Action
Short Description Validation	Short Description empty	Display error
Resolution Code Policy	State = Resolved	Make Resolution Code mandatory
Close Notes Policy	State = Resolved	Make Close Notes mandatory
Assignment Policy	Assignment Group selected	Control Assigned To behavior
Category-based Policy	Category selected	Control relevant fields
Impact/Urgency Validation	Values changed	Validate Incident information

These examples demonstrate how ServiceNow can dynamically control Incident forms.

14. TESTING PLAN

Testing is performed after implementing Client Scripts and UI Policies.

The following test cases can be used:

Test Case	Input/Condition	Expected Result
TC01	Open new Incident	Incident form loads correctly
TC02	Submit without Short Description	Validation message displayed
TC03	Enter valid Short Description	Form accepts the value
TC04	State changed to Resolved	Resolution fields become mandatory
TC05	Required fields left empty	Submission is restricted
TC06	Valid required information entered	Incident can be submitted
TC07	Change configured field	Related field behavior changes
TC08	Open existing Incident	Configured policies work correctly

15. EXPECTED RESULTS

After successful implementation:

Incident forms should provide better validation.

Required fields should be identified automatically.

Relevant fields should appear according to conditions.

Unnecessary fields can be hidden.

Restricted fields can be made read-only.

Incorrect submissions can be prevented.

Incident data quality should improve.

Users should have a clearer and more consistent form experience.

16. ADVANTAGES OF THE PROJECT

Improved Data Accuracy

Client-side validation reduces incorrect or incomplete information.

Better User Experience

Users see only relevant fields and receive immediate feedback.

Reduced Manual Work

Dynamic form behavior reduces the need for manual verification.

Improved Incident Management

Better incident information helps support teams process incidents efficiently.

Easy Customization

ServiceNow allows administrators to customize forms according to organizational requirements.

Maintainability

UI Policies can handle many basic field-behavior requirements without writing extensive JavaScript.

17. PROJECT LIMITATIONS

The project has some limitations:

Client Scripts mainly control behavior on the client side.

Client-side validation should not be considered a replacement for server-side validation.

Incorrect configuration can cause unexpected form behavior.

Complex requirements may require additional ServiceNow scripting or configuration.

The project focuses specifically on the Incident application.

18. FUTURE ENHANCEMENTS

The project can be enhanced in the future by:

Adding more advanced validation rules.

Integrating Incident Management with other ServiceNow modules.

Adding automated notifications.

Creating dashboards and reports.

Implementing automated workflows.

Adding role-based form customization.

Integrating external systems.

Using additional ServiceNow automation features.

Improving incident analytics and monitoring.

19. PROJECT OUTCOME

The project demonstrates how Client Scripts and UI Policies can be used to customize the Incident application in ServiceNow.

Client Scripts provide customized client-side logic and validation, while UI Policies provide dynamic control over field visibility, mandatory status, and read-only behavior.

The implementation helps create a more structured Incident form, improves the quality of information entered by users, and provides a better overall user experience.

20. CONCLUSION

The Implementation of Client Scripts and UI Policies in the Incident Application using ServiceNow provides practical knowledge of ServiceNow customization and Incident Management.

Through this project, the behavior of Incident forms can be dynamically controlled based on user actions and specific conditions. Client Scripts can be used for validation and customized client-side logic, while UI Policies can manage field visibility, mandatory status, and read-only behavior.

The project provides hands-on experience in analyzing requirements, configuring ServiceNow components, implementing scripts and policies, testing different scenarios, and documenting the final implementation.

Overall, the project demonstrates how ServiceNow configuration features can be used to improve data accuracy, form usability, consistency, and efficiency in Incident Management.